601.433/633 Introduction to Algorithms **Fall 2026**
Homework #1 **Due:** September 17, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Asymptotic Notation (25 points)

For each of the following statements say if it true or false and prove your answer. The base of log is 2 unless otherwise specified, and ln is $\log_e$.

(a) $n \tan n = O(2^n)$

We can define function $n \tan n$ as $O(2^n)$ if there exist constants c, $n_0 > 0$ for which $n \tan n \leq c * 2^n$ for all $n > n_0$. We must assume n is an integer greater than 0 and c is a real number.

Let us look at the function $n \tan n$. Since n must be a real number and n_0 must be greater than 0, we should examine the range of possible values of function $n \tan n$. $n \tan n$ has a vertical asymptote whenever $n = \frac{k\pi}{2}$ where k is any odd integer. The limit of $n \tan n$ as n approaches $\frac{k\pi}{2}$ is ∞. Therefore, for any possible real value of c and n_0, at the next value of n where $n > n_0$ and $n = \frac{k\pi}{2}$ for some k, $n \tan n$ would be so close to ∞ that it could not be less than or equal to $c * 2 * n$ for any c.

Therefore, the statement is false

(b) $e^n = \Theta(2^n)$

We can define function e^n as part of the set of $\Theta(2^n)$ functions if e^n satisfies both $\Omega(2^n)$ and $O(2^n)$.

Let us first consider $e^n = \Omega(2^n)$. We must find constants c and $n_0 > 0$ for which $e^n \geq c * 2^n$ for all $n > n_0$. We will set c to 1, yielding the inequality:

$e^n \geq 2^n$

Since $e > 2$, we can see that $e^n \geq 2^n$ will hold for all n where $n \geq 1$, so we will set n_0 to 1. Since we have found constants c and n_0 that satisfy $e^n \geq c * 2^n$, we have proven that $e^n = \Omega(2^n)$.

Let us move on to $e^n = O(2^n)$. We must find constants c and $n_0 > 0$ for which $e^n \leq c * 2^n$ for all $n > n_0$. However, $e > 2$. For any real constant c, for some n, e^n will surpass 2^n. I will use the general unsigned intmax value, 2^{32} to illustrate this.

$e^n \leq 2^{32} * 2^n$

$e^n \leq 2^{n+32}$

e can be expressed roughly as $2^{1.442}$. Taking this, we get the inequality:

$2^{1.442n} \leq 2^{n+32}$

Thus, the value for n at which the left side of the equation surpasses the right is roughly $n = 72.4$. The same pattern applies to any possible value for c, as e^n grows at a different order of magnitude as 2^n. Therefore, $e^n = O(2^n)$ is false.

This means that $e^n = \Theta(2^n)$ is false.

(c) $n \cos n = O(n)$

We can define function $n \cos n$ as $O(n)$ if there exist constants c, $n_0 > 0$ for which $n \cos n \leq cn$ for all $n > n_0$. We must assume n is an integer greater than 0 and c is a real number. Let us set c to 2, and we can solve the simple inequality of:

$n \cos n \leq 2n$

$(n \cos n)/n \leq 2$

$\cos n \leq 2$

Since the maximum value of $\cos n$ for any n is 1, the inequality $\cos n \leq 2$ holds for any integer n, so we set n_0 to 1. Since we have found constants c and n_0 for which the inequality $n \cos n \leq cn$ holds for all $n > n_0$, we have proven that $n \cos n = O(n)$.

Therefore, the statement is true.

(d) $3^n = \Omega(3^{(n+2)})$

We can consider function 3^n as part of the set of $\Omega(3^{(n+2)})$ if there exist constants c, $n_0 > 0$ for which $3^n \geq c * 3^{(n+2)}$ for all $n > n_0$. We must assume n is an integer greater than 0 and c is a real number. Let us set c to $\frac{1}{27}$, which gives the following inequality:

$3^n \geq \frac{1}{27} * 3^{(n+2)}$

This can be rewritten as:

$3^n \geq 3^{-3} * 3^{(n+2)}$

$3^n \geq 3^{(n+2)-3}$

$3^n \geq 3^{(n-1)}$

Since n_0 must be a positive value greater than 0, we can set $n_0 = 1$, giving the inequality:

$3^1 \geq 3^{(1-1)}$

$3 \geq 1$

This pattern continues, since as n increases, the value of $\frac{1}{27} * 3^{(n+2)}$ remains exactly $\frac{1}{3}$ of 3^n, which means $3^n \geq c * 3^{(n+2)}$ is satisfied and $3^n = \Omega(3^{(n+2)})$.

Therefore, the statement is true.

(e) $\log(n^{\frac{1}{5}}) = \Theta(\log(n^3))$

We can define function $\log(n^{\frac{1}{5}})$ as part of the set of $\Theta(\log(n^3))$ functions if $\log(n^{\frac{1}{5}})$ satisfies both $\Omega(\log(n^3))$ and $O(\log(n^3))$.

Let us first consider $\log(n^{\frac{1}{5}}) = \Omega(\log(n^3))$. We must find constants c and $n_0 > 0$ for which $\log(n^{\frac{1}{5}}) \geq c * \log(n^3)$ for all $n > n_0$. Let us set c to $\frac{1}{18}$, which yields the inequality:

$\log(n^{\frac{1}{5}}) \geq \frac{1}{18} * \log(n^3)$

$\log(n^{\frac{1}{5}}) \geq \log((n^3)^{\frac{1}{18}})$

$\log(n^{\frac{1}{5}}) \geq \log(n^{\frac{1}{6}})$

Since $\frac{1}{5} \geq \frac{1}{6}$ for any n where $n \geq 1$, then $n^{\frac{1}{5}} \geq n^{\frac{1}{6}}$. Therefore, $\log(n^{\frac{1}{5}}) \geq \log(n^{\frac{1}{6}})$, and n_0 can be set to 1. Since we have found constants c and n_0 for which $\log(n^{\frac{1}{5}}) \geq c * \log(n^3)$ for all $n > n_0$, we have proven that $\log(n^{\frac{1}{5}}) = \Omega(\log(n^3))$.

Let us move on to $\log(n^{\frac{1}{5}}) = O(\log(n^3))$. We must find constants c and $n_0 > 0$ for which $\log(n^{\frac{1}{5}}) \leq c * \log(n^3)$ for all $n > n_0$. Let us set c to 1, which yields the inequality:

$\log(n^{\frac{1}{5}}) \leq \log(n^3)$

A similar logic to the above applies, where $\frac{1}{5} \leq 3$, so $n^{\frac{1}{5}} \leq n^3$, when we consider n where $n \geq 1$. Therefore, we set n_0 to 1, and we have found constants c, n_0 which satisfy $\log(n^{\frac{1}{5}}) \leq c*\log(n^3)$. Thus, $\log(n^{\frac{1}{5}}) = O(\log(n^3))$.

In proving both $\log(n^{\frac{1}{5}}) = \Omega(\log(n^3))$ and $\log(n^{\frac{1}{5}}) = O(\log(n^3))$, we prove $\log(n^{\frac{1}{5}}) = \Theta(\log(n^3))$.

Thus, the statement is true.

(f) Let f, g be positive functions. Then $f(n) + g(n) = \Omega(\max(f(n), g(n)))$

We can consider function $f(n) + g(n)$ as part of the set of $\Omega(\max(f(n), g(n)))$ if there exist constants c, $n_0 > 0$ for which $f(n) + g(n) \geq c * \max(f(n), g(n))$ for all $n > n_0$. Since f and g are positive functions, all values of $f(n)$ and $g(n)$ are positive. Let us set c to 1, which gives the following inequality:

$f(n) + g(n) \geq \max(f(n), g(n))$

Since $f(n)$ and $g(n)$ are both positive for any n, their sum must be greater than either $f(n)$ or $g(n)$. Thus, the sum of $f(n)$ and $g(n)$ is greater than the maximum between the two. Thus, the inequality holds for all n, so we can set n_0 to 1. Since we have found constants c and n_0 for which $f(n) + g(n) \geq \max(f(n), g(n))$ holds, we have proven that $f(n) + g(n) = \Omega(\max(f(n), g(n)))$.

Therefore, the statement is true.

2 Recurrences (25 pts)

Solve the following recurrences, giving your answer in Θ notation (so prove both an upper bound and a lower bound). For each of them you may assume $T(x) = 1$ for $x \leq 5$. Justify your answer (formal proof not necessary, but recommended).

(a) $T(n) = 5T(n-3)$

We can begin by expanding the recurrence to write $T(n)$ in terms of $T(n-6)$ and $T(n-9)$ to establish a pattern:

$T(n) = 5T(n-3)$

$T(n) = 5(5T(n-6))$

$T(n) = 5^2T(n-6)$

$T(n) = 5^3T(n-9)$

This creates the pattern:

$T(n) = 5^kT(n-3k)$ for some positive integer k.

Let us know build out the base case. We know that $T(1) = 1$. Using the definition of big-O, we know that to prove $T(1) = O(1)$, we must find constants c and n_0 for which $T(1) \leq c * 1$ for all $n > n_0$. Since $T(1)$ is 1, using $c = 1$, we can see that $T(1) \leq 1$ for all n. Thus, $T(1) = O(1)$.

The same logic applies for proving big-Omega. We must find constants c and $n_0 > 0$ for which $T(1) \geq c * 1$ for all $n > n_0$. Since $T(1)$ is 1, using $c = 1$, we can see that $T(1) \geq 1$ for all n. Thus, $T(1) = \Omega(1)$.

Since $T(1)$ is both $O(1)$ and $\Omega(1)$, we have proven that $T(1) = \Theta(1)$.

Based on the above equation I solved, our base case is reached when $n-3k = 1$. Solving this, we get:

$k = \frac{n-1}{3}$

Substituting this back into the exanded recurrence, now that we know that the $n - 3k = 1$:

$T(n) = 5^{\frac{n-1}{3}}T(1)$

$T(n) = 5^{\frac{n-1}{3}} = 5^{\frac{n}{3}}5^{\frac{-1}{3}}$

$T(n) = 5^{\frac{n-1}{3}} = (5^{\frac{1}{3}})^n5^{\frac{-1}{3}}$

Since $5^{\frac{-1}{3}}$ is a constant, we can set it to c and write $T(n)$:

$T(n) = c * (5^{\frac{1}{3}})^n$

Since we can express $T(n)$ as a constant multiplied by an expression involving n, we have found the growth rate of $T(n)$. In finding the growth rate, we can express $T(n)$ in big-Theta notation.

Therefore, $T(n) = \Theta((5^{\frac{1}{3}})^n)$

(b) $T(n) = n^{1/4}T(n^{3/4}) + n$

We will begin with expanding the recurrence like I did in the last problem to find a general pattern:

$T(n) = n^{1/4}T(n^{3/4}) + n$

$T(n^{3/4}) = n^{3/16}T(n^{9/16}) + n^{3/4}$

$T(n) = n^{1/4}(n^{3/16}T(n^{9/16}) + n^{3/4}) + n$

$T(n) = n^{1/4}(n^{3/16}T(n^{9/16})) + 2n$

$T(n^{9/16}) = n^{9/64}T(n^{27/64}) + n^{9/16}$

$T(n) = n^{1/4}(n^{3/16}(n^{9/64}T(n^{27/64}) + n^{9/16})) + 2n$

$T(n) = n^{1/4}(n^{3/16}n^{9/64}T(n^{27/64}) + n^{3/4}) + 2n$

$T(n) = n^{1/4}n^{3/16}n^{9/64}T(n^{27/64}) + 3n$

We can write a general equation with the following pattern:

$T(n) = (n^{(1/4)\sum_{i=0}^{k-1}((3/4)^i)})T((n^{3/4})^k) + kn$ where k is any integer > 0.

We can solve the geometric sequence to yield:

$\sum_{i=0}^{k-1}((3/4)^i) = \frac{1(1-(\frac{3}{4})^k)}{1-\frac{3}{4}} = 4(1-(\frac{3}{4})^k)$

Let us input this term back into the equation to yield:

$T(n) = (n^{(1-(\frac{3}{4})^k)})T((n^{3/4})^k) + kn$

Let us now look at the term $T((n^{3/4})^k)$. We should look to the base case, where $T((n^{3/4})^k) = T(1)$. We know that $T(1) = \Theta(1)$ by the proof in part (a) and in understanding growth rates. For ease of computation, instead of looking at $T(1)$, we can look at $T(2)$, which is also 1 as per the rules of this question. Therefore, to solve for k, we must solve:

$(n^{3/4})^k = 2$

Taking the log base 2 of both sides, we get:

$\log_2(n^{3/4})^k = \log_2 2$

$(\frac{3}{4})^k \log_2 n = \log_2 2$

$(\frac{3}{4})^k = \frac{1}{\log_2 n}$

Now, we have a growth rate for $(\frac{3}{4})^k$, so we can insert the resulting expression back into the original equation for $T(n)$:

$T(n) = (n^{(1-\frac{1}{\log n})})T((n^{3/4})^k) + kn$

Since $T((n^{3/4})^k) = T(2) = 1$:

$T(n) = (n^{(1-\frac{1}{\log n})}) + kn$

$T(n) = (n * n^{-\frac{1}{\log n}}) + kn$

Since $n = 2^{log_2 n}$, we can rewrite $n^{-\frac{1}{\log_2 n}}$ as:

$(2^{log_2 n})^{-\frac{1}{\log_2 n}} = 2^{-\frac{\log_2 n}{\log_2 n}} = 2^{-1} = \frac{1}{2}$

Therefore:

$T(n) = (\frac{n}{2}) + kn$

We can once again look at the formula for k and solve further for k:

$(\frac{3}{4})^k = \frac{1}{\log_2 n}$

$k = \log_{\frac{3}{4}}(\frac{1}{\log_2 n})$

$k = \log_{\frac{3}{4}}(1) - \log_{\frac{3}{4}}(\log_2 n)$

$k = 0 - \log_{\frac{3}{4}}(\log_2 n)$

$k = -\log_{\frac{3}{4}}(\log_2 n)$

$k = \log_{\frac{4}{3}}(\log_2 n)$

Therefore, we can rewrite the equation as:

$T(n) = (\frac{n}{2}) + n \log_{\frac{4}{3}}(\log_2 n)$

Since changing the base of a logrithm changes it by a constant factor, and $\frac{1}{2}$ is a constant, we can rewrite this equation as:

$T(n) = c_1 * n + c_2 * n \log\log n$

Therefore, we know the growth rate of the equation, so we can express it as:

$T(n) = \Theta(n + n \log\log n)$

However, $\log\log n$ grows without bound as n increases, so $n \log\log n$ eventually surpasses n, meaning it dominates the equation. Therefore, we can conclude that $T(n) = \Theta(n \log\log n)$

(c) $T(n) = 6T(n/4) + n$

Here, we can apply the Master Theorem, which states:

$T(n) = a * T(n/b) + cn^k$

In this case, $a = 6$, $b = 4$, $c = 1$, and $k = 1$.

We let $\alpha = a/b^k$. For this relation, $\alpha = 6/4 = 1.5$

Then, using the master theorem, we can write $T(n)$ as:

$T(n) = cn^k(1 + \alpha + \alpha^2 + ... + \alpha^{\log_b n})$ $T(n) = n(1 + 1.5 + 1.5^2 + ... + 1.5^{\log_4 n})$

Using the master theorem, we consider the case where $\alpha > 1$. Looking at $(1 + 1.5 + 1.5^2 + ... + 1.5^{\log_4 n})$, we can write the following inequality:

$(1 + 1.5 + 1.5^2 + ... + 1.5^{\log_4 n}) \leq 1.5^{\log_4 n}(\frac{1}{1-\frac{1}{1.5}})$

This means:

$1.5^{\log_4 n}(\frac{1}{1-\frac{1}{1.5}}) = O(1.5^{\log_4 n})$

We can conclude that the $(1+1.5+1.5^2+...+1.5^{\log_4 n})$ term dominates the overall recurrence relation, so we can write it as:

$T(n) = \Theta(n * 1.5^{\log_4 n}) = \Theta(n * (6/4)^{\log_4 n})$

Noting that $4 \log_4 n = n$, we can conclude that:

$T(n) = \Theta(n^{log_4 6})$

3 Group Sorting (50 points)

We say that an array A of size n is *k-group sorted* if it can be divided into k consecutive groups, each of size n/k, such that the elements in each group are larger than the elements in earlier groups, and smaller than elements in later groups. The elements within each group need not be sorted.

For example, the following array is 4-group sorted:

1	2	4	3	7	6	8	5	10	11	9	12	15	13	16	14

Note that every array is 1-group-sorted, and only sorted arrays are n-group sorted. You may assume that all elements are distinct, and if you want to you may assume that n and k are powers of 2.

Describe an algorithm that k-group-sorts an array in (worst-case) time $O(n \log k)$. In other words, in at most $O(n \log k)$ time it must turn an array which is not k-group sorted into one that is. Prove correctness and running time.

Algorithm:

I assume n and k are powers of 2. I will use a modified version of quicksort for this algorithm. I use the BFPRT algorithm described in class to find the most efficient pivot point, which takes $O(n)$ time. From here, I will recursively perform partitions of the list so that the first level finds the median of the array and splits it into 2 equally-sized groups, the second finds the median of each of the two groups and splits those in half, yielding 4 groups, the same pattern repeats to create 8 groups, and the algorithm continues until the array is divided into k groups. Each level of the algorithm doubles the number of groups, so it there would need to be $\log k$ calls. For each call, the BFPRT algorithm would be performed, and then each element of the list would be passed over once as the partition occurs, meaning each call takes $O(n)$ time. This leads to $O(n \log k)$ time, and creates a k-group sorted array.

Correctness proof:

We use proof by basic induction.

Base case: array A is 1-group sorted. Since every group is 1 group sorted, the base case holds.

Inductive step: We seek to prove that if array A is k-group sorted and we apply the algorithm, then it will be $2k$-group sorted.

Assume A is k-group sorted. We will prove that A becomes $2k$-group sorted after applying this algorithm. Let us focus on one group in A. We will call this group G. Using the proof of the BFPRT algotihm, we know the pivot point of G returned is the median of G. BFPRT finds an element that allows G to be partitioned into two equal-sized groups, with every element in the left group smaller than every element in the right group. We swap the median pivot with the last element. From here, we create two pointers, a left pointer and a right pointer. The left pointer is initially set to the first element in G, while the right is set to the second-to-last (since the last is the pivot). From here, the algorithm examines the left pointer, and determines if it is greater or less than the pivot. If the element is less than the pivot point, it remains in place and the left pointer advances. If the element is greater than the pivot point, the right pointer is checked, and it decreases by 1 until the pointer points to an element smaller than the pivot. This continues until the left pivot passes the right pivot. Then, the pivot point swaps with the left pivot. Since the element swapped has already been examined by the right pointer, we know that it does belong on the right side of the equation. Now, we know that every element in the left half is smaller than the right half. Therefore, when the partition occurs at the middle index of G, creating groups G_{left} and G_{right}, we know that for all elements e in G_{left}, e is less than the pivot point, and for all elements e in G_{right}, e is greater than the pivot point. This process is applied independently to each of the k groups in A. Each group is divided into 2 groups such that every element in the left group is smaller than the right. Furthermore, since A is already k-group sorted, we know that every element in any group G_i is smaller than the elements in the group to the right G_{i+1}. Therefore, the resulting array after partitioning is $2k$-group sorted, and the inductive step holds.

In proving that the base case and the inductive hypothesis both hold, I have proven the correctness of my algorithm.

Runtime proof:

I will first prove that each level of the call to the algorithm to double the number of groups in the list requires $O(n)$ time, where n is the number of elements in array A.

The BFPRT algorithm has $O(n)$ time complexity, where n is the number of elements in the group it is finding the median of, as per the proof of the BFPRT algorithm time complexity in the class slides. When the array is k-group sorted, the BFPRT algorithm repeats k times, where each scan goes over the $\frac{n}{k}$ elements in each group, as per the division of the groups. Therefore, using a simple calculation, we can find that on each level of computation, BFPRT takes $O(n)$ times.

Now, I will examine the partition algorithm. This algorithm examines each element of the array and performs swaps. Each swap can be broken down to $O(1)$ time, since the swap is constant no matter the number of elements in the list. The partition algorithm examines each element a constant number of times, and each swap takes $O(1)$ time. As per the proof of algorithmic correctness, we can see that the partition algorithm keeps track of two pointers, which advance through the list until the left pointer surpasses the right. This means that each element of the group is examined once. We know that each group of a k-group sorted array has $\frac{n}{k}$ elements, and there are k groups. Since the partition must run for each group, this means there are k runs of an $O(\frac{n}{k})$ algorithm, meaning that for each round of partitions, where the number of groups in the array doubles, $O(n)$ work is done.

We can see that the BFPRT algorithm and the partition algorithm have an upper bound of $O(2n)$ together, which simplifies to $O(n)$.

Now, I will prove that the BFPRT + partition algorithm to double the number of groups by which an array is sorted by must run $\log k$ times to fully split the array into k groups.

As established in the correctness proof, each call to the BFPRT + partition algorithm doubles the number of groups. On the first iteration, the number of groups goes from 1 to 2. The next time, it goes from 2 to 4. This creates the pattern that after i iterations, there are 2^i groups. Therefore, to reach k groups, we can set up the simple equivalence:

$k = 2^i$

$i = \log_2 k$

Therefore, to create a k-group sorted array, the number of iterations required is $\log k$.

Therefore, since I have proven that there are $\log k$ calls to an algorithm that takes $O(n)$ time, we can conclude that this k-group sorting algorithm has a worst-case time of $O(n \log k)$.